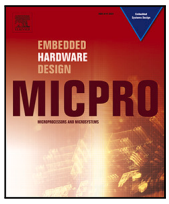
Microprocessors and Microsystems 122 (2026) 105258

Contents lists available at ScienceDirect

Microprocessors and Microsystems

journal homepage: www.elsevier.com/locate/micpro



CGR-AI Engine: A scalable CGRA-based processing platform for Artificial Intelligence in space applications☆

Luca Zulberti [a], Matteo Monopoli [a,b,*], Gabriela Mystkowska [a], Andrea Monorchio [a], Silvia Moranti [b], Luca Fanucci [a], Pietro Nannipieri [a]

[a] University of Pisa, Department of Information Engineering, Via G. Caruso 16, Pisa, 56122, Italy
[b] European Space Agency, European Space Research and Technology Centre (ESTEC), Keplerlaan 1, Noordwijk, 2201 AZ, The Netherlands

ARTICLE INFO

Keywords:
CGRA
Space
Artificial Intelligence
RISC-V
Hardware acceleration
Standard cell
Convolutional Neural Network

ABSTRACT

The execution of processing tasks in space is a cornerstone of modern space missions, enabling a wide range of critical applications. In recent years, the integration of Artificial Intelligence (AI) within the space industry has gained prominence, driven by its superior performance compared to traditional methods, particularly in image processing. This trend has accelerated the adoption of AI in the space sector. In this context, the choice of processing architecture onboard spacecraft becomes crucial for ensuring the efficiency of inference algorithms and maximizing platform versatility across diverse missions. This paper introduces the CGR-AI Engine, a scalable Coarse-Grained Reconfigurable Array (CGRA)-based processing platform designed to facilitate the deployment of AI applications in space. The platform comprises highly customizable components, whose capabilities can be optimized through Design Space Exploration (DSE) activities. The results of these optimizations are presented in this work. By combining the programmability of a RISC-V processor, employed as a memory transaction scheduler, with the reconfigurability of a CGRA processing matrix for implementing Data Flow Graphs (DFGs) of specific algorithms, the CGR-AI Engine offers a flexible and adaptable solution. It is well-suited for diverse workloads and can be tailored to meet specific application requirements, as well as stringent resource and power constraints. This paper outlines the methodology for implementing general-purpose Neural Network (NN) layers and models. It presents synthesis outcomes on the DARE65T standard cell library platform and provides a detailed analysis of benchmarks conducted to assess their performance, validating the proposed solution's capability to meet the demanding requirements of onboard processing in space missions.

1. Introduction

AI algorithms have shown significant value [1,2] for navigation systems, land and marine monitoring, climate and meteorological observation, star tracking, attitude control, object tracking, and event detection. NNs play a pivotal role in data processing within the space domain due to their ability to analyze large-scale, high-dimensional data, such as satellite imagery and remote sensing data [3,4]. Feature recognition and extraction techniques, powered by NNs, enable more efficient analysis of satellite imagery by reducing the bandwidth requirement of satellite-ground communication [5], i.e., in multispectral and hyperspectral image analysis [6,7].

In recent years, there has been a significant increase in interest regarding the use and deployment of AI technologies directly onboard satellites. The ϕ-Sat-1 mission from the European Space Agency (ESA) was a pioneering effort in this direction, demonstrating the feasibility and potential of onboard Machine Learning (ML) by running a cloud detection algorithm on the Myriad 2 Vision Processing Unit (VPU) [7]. Since then, research has explored the advantages of using NNs for applications such as Earth Observation (EO) and remote sensing. Notably, the OPS-SAT CubeSat has been employed for in-orbit training and Fault Detection Isolation and Recovery (FDIR), marking another milestone in onboard AI experimentation [8–10]. Despite growing interest from both academia and industry [11,12], real-world deployments of NNs in

☆ This article is part of a Special issue entitled: 'DSD 2024' published in Microprocessors and Microsystems.

* Corresponding author at: University of Pisa, Department of Information Engineering, Via G. Caruso 16, Pisa, 56122, Italy.
E-mail addresses: luca.zulberti@phd.unipi.it (L. Zulberti), matteo.monopoli@phd.unipi.it (M. Monopoli), gabriela.mystkowska@phd.unipi.it (G. Mystkowska), andrea.monorchio@phd.unipi.it (A. Monorchio), silvia.moranti@esa.int (S. Moranti), luca.fanucci@unipi.it (L. Fanucci), pietro.nannipieri@unipi.it (P. Nannipieri).
URL: https://vlsi.iet.unipi.it (L. Zulberti).

https://doi.org/10.1016/j.micpro.2026.105258
Received 15 January 2025; Received in revised form 11 November 2025; Accepted 2 March 2026
Available online 3 March 2026
0141-9331/© 2026 Elsevier B.V. All rights are reserved, including those for text and data mining, AI training, and similar technologies.

Table 1
Comparison of SoA CGRA Solutions.

Solution	Strengths	Limitations
ML-CGRA [32]	MLIR integration, optimized ML templates	Dependent on pre-configured models, limited scalability
ASAP [35]	Multi-precision support, automated DSE	Limited precision in high-performance contexts
APEX [38]	Energy and area efficiency, optimized for specific tasks	Limited to specific application domains
PX-CGRA [39]	Flexibility with approximate computations	Loss of precision for critical tasks
Eyeriss v2 [40]	Optimized for DNNs, support for sparsity	Architectural limitations in non-DNN tasks
Fifer [41]	Optimization for irregular applications	Requires complex dynamic pipelines
PICACHU [42]	Optimization for nonlinear operations in LLMs	Limited scalability of CGRA mapping

3. CGR-AI engine platform

This Section presents the CGR-AI Engine, a highly parameterizable platform based on CGRA for the acceleration of Digital Signal Processing (DSP) algorithms and NNs. It comprises data-moving elements, memory blocks, a CGRA processing core, and a RISC-V CPU to orchestrate data movement inside the architecture. The CGR-AI Engine executes kernels that schedule memory transactions, loading data from external memory to local memory, from local memory to embedded CGRA matrix, and vice versa. Kernels can map DFGs to available CGRA resources, adjusting the parallelism of the algorithm that is being executed. At this stage, no dedicated compiler has been developed; therefore, the mapping of each DFG node to a specific operation is performed statically. However, kernels can dynamically configure both the operation assigned to each FU and their interconnections within the PE by specifying the configuration directly in the C code executed by the host processor. Scheduling can be based on runtime constraints such as memory bandwidth and power consumption. The CGR-AI Engine is fully written in VHDL, and its parameters can be tuned depending on area, power, and performance requirements. Fig. 1 illustrates the proposed platform's block diagram, emphasizing the CGR-AI Engine's key components. These include the CGRA processing core, previously detailed in [43–45], a custom RISC-V CPU, the memory subsystem featuring Memory-Mapped to Streaming (MM2S) and Streaming to Memory-Mapped (S2MM) DMEs, banked local memories, the Events Switch, and a Kernel Management Unit (KMU).

3.1. Architecture overview and topology

The modules composing the CGR-AI Engine account for three different main tasks:

- **Platform Management**: the host accesses control and status registers through a dedicated Memory-Mapped (MM) slave interface. They are handled by the *Kernel Management Unit*, which is responsible for kernel loading/start/stop and communication with the host in general.
- **Algorithm Scheduling**: the platform is controlled by a custom RISC-V CPU, whose firmware is responsible for dispatching data-moving commands to DMEs and configuring the CGRA matrix. DMEs are dedicated modules to issue memory requests in the architecture.
- **Data Processing**: data moved by the DMEs reaches the CGRA processing blocks, whose results are stored temporarily in local memory before being written back to memory or processed again by the CGRA.

From a data processing perspective, the CGRA core supports fixed-point operations through FUs integrated within each tile. Up to six FUs can be instantiated per tile to perform operations such as addition (ADD), multiplication (MUL), shifting (SHF), and bitwise logic (BTW), as well as to implement multiplexers (MUX) and look-up tables (LUT). These FUs are optimized for integer arithmetic, which is well-suited for inference workloads in quantized neural networks. By focusing on low-bitwidth quantized operations (e.g., INT8 and INT16), the architecture enables efficient execution of deep learning models while minimizing power and area overhead. This makes the CGRA particularly effective for embedded AI applications where real-time inference is required under tight resource constraints. Tiles are interconnected using a simple ready/valid handshake protocol, enabling efficient data streaming between them.

The interconnection topology can range from a basic row-wise configuration to more complex schemes that include vertical and diagonal links, depending on the latency requirements and the structure of the DFGs to be implemented. For our target application, we opted for a straightforward interconnect between tiles, where rows operate independently without inter-row communication. Nonetheless, multiple rows can still be employed to provide redundancy if required. A brief overview of the CGRA core is presented in Fig. 2.

From the system perspective, the functional model of the platform can be summarized as follows: Firstly, the host must configure the platform by communicating with the KMU to instruct the device to run a particular kernel. At that point, the KMU exploits the DMEs, connected between the low-speed reading port and the Tightly-Coupled Memory (TCM) to load the kernel onboard. Once loaded, the kernel comprising the RISC-V firmware and CGRA configurations, is executed by waking up the CPU.

Secondly, the firmware running on RISC-V exploits dedicated instructions to drive all DMEs connected between high-speed reading ports and the banked local memory to load data onboard. Once data is loaded, the firmware configures the CGRA using a Configuration DME (CDME), making it ready to process the data. After that, it drives the DMEs responsible for taking the data to the CGRA processing chain and returning the results to local memory. This operation is performed until new data are needed or local memory must be flushed back to external memory.

Lastly, the write-back operation is demanded to the DMEs connected between the high-speed writing ports and the banked local memory. The Events Switch is used to synchronize all different memory transactions.

The proposed system is highly configurable to fit the available memory bandwidth in the integrated system. As highlighted in Fig. 1, many parameters can change the resulting area occupation, the power efficiency, the performance, and the versatility of the CGR-AI Engine:

- `CGRA_ROWS`. Each row is identical and can be allocated for parallel computations. The number of rows determines also the number of DMEs (2× MM2S and 1× S2MM per row), hence the number of ports of the banked local memory. Although the number of columns in the CGRA core is configurable, the Engine adopts a 2-tile-per-row configuration, which is the most efficient for executing convolutional layers in CNNs, specifically, *Conv2D* operations followed by activation functions.
- `TCM_*`. The CPU memory can be sized in terms of depth (`_ADDR_WIDTH`), the number of banks (`_BANKS_COUNT`), and bank groups (`_BANKS_GROUP`).
- `LMEM_*`. Same as the CPU memory, but it can be configured also in terms of data width (`_DATA_WIDTH`).

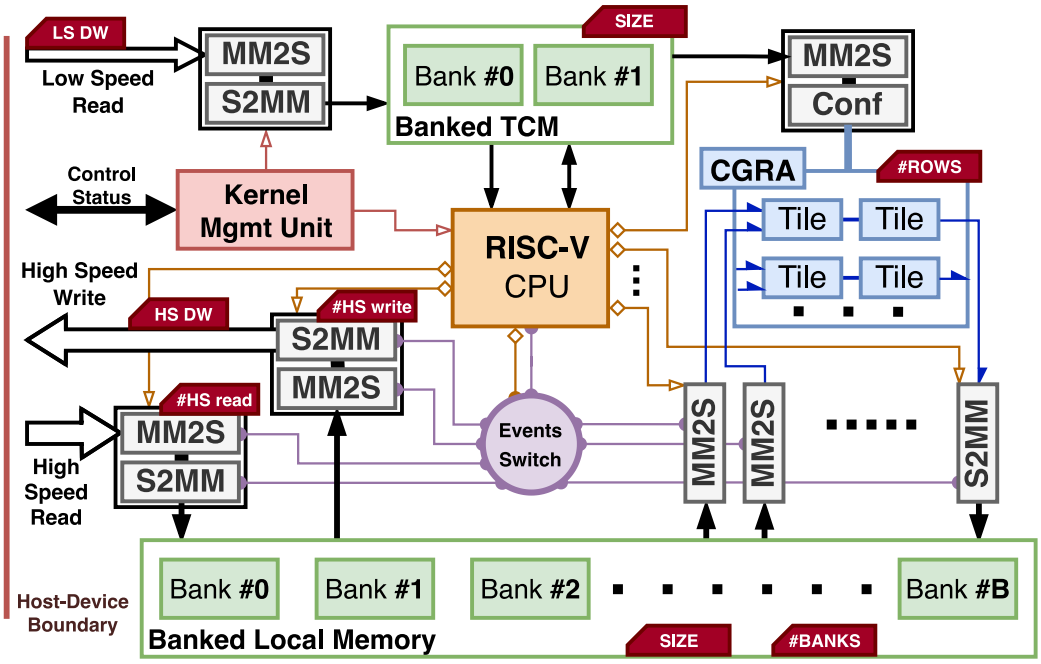


Fig. 1. The CGR-AI Engine is composed of several elements: one RISC-V core, one processing CGRA core, reading and writing High-Speed (HS) DMEs (MM2S-S2MM combos), one reading Low-Speed (LS) DME combo, one Kernel Management Unit, two banked memories (local memory for data and TCM for firmware), and several DMEs for data movement between local memory and CGRA. The platform parameters are highlighted in red boxes, from the top-left corner: Low-Speed Data-Width, TCM Size, number of CGRA Rows, High-Speed Data-Width, number of High-Speed reading and writing ports, local memory Size and number of Banks. (For interpretation of the references to color in this figure legend, the reader is referred to the web version of this article.)

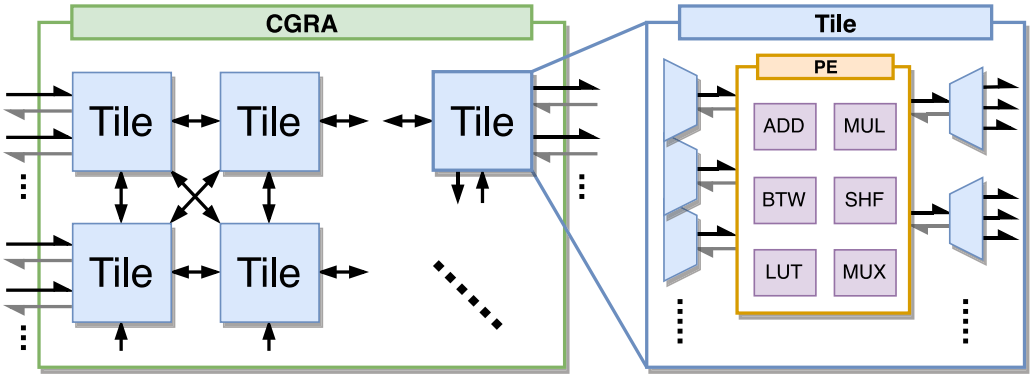


Fig. 2. CGRA core architecture where each tile contains a PE composed of basic functional units such as ADD, MUL, LUT, SHF, and MUX. The interconnection between tiles supports flexible data routing and parallel computation.

- `HS_*`. High-Speed memory-mapped interfaces can be configured in terms of the number of reading and writing ports (`_RD/WR_PORTS_COUNT`), address and data widths (`_ADDR/DATA_WIDTH`).
- `KERNEL_*`. Same as the High-Speed ports, but it is a single reading port with configurable address and data widths (`_ADDR/DATA_WIDTH`).

3.2. Programming model

CGRAs require the programmer to think about data flow, parallelism, and spatial mapping of tasks to the architecture. In particular, since operations are executed in parallel across different FUs, the scheduling algorithm must efficiently allocate resources and dispatch memory transactions to maximize performance. This involves determining which Tile of the CGRA matrix will execute which operations and how data will be routed between them. The programming model of this architecture is the dataflow model, where the execution is driven by the availability of data rather than by a sequential instruction stream. The CGR-AI Engine also implements a pipelined parallelism, where different stages of a computation pipeline can be mapped onto different Tiles and executed concurrently.

To simplify the mapping and scheduling process, the CGRA matrix is designed to execute one job per row, while the columns are needed for pipelined parallelism. DFGs of a certain algorithm can be mapped on multiple matrix rows to increase performance by exploiting data reuse. Another aspect of the dataflow programming model is the efficiency in performing memory accesses, particularly in applications with irregular memory access patterns [46]. Using DMEs, the firmware on the RISC-V can implement custom strategies to pre-fetch data and load complex patterns into CGRA from local memory.

Two programming Application Programming Interfaces (APIs) have been developed for the CGR-AI Engine, one for the host and one for the device.

The host programming API is pretty similar to the programming API of GPUs. A kernel, composed of the scheduling firmware and a set of CGRA configurations, must be loaded into the device. Kernel parameters, such as the source address of the input data and the destination address of results, can be set within the KMU. When the kernel starts, it can load those parameters to customize its execution. The CGR-AI Engine communicates with the host by exploiting interrupt lines. When the kernel terminates its execution or other events occur (e.g., errors), an interrupt is fired, and its cause can be determined by reading the status register from the KMU.

The device programming API is based on a set of primitives that implement a custom RISC-V Instruction Set Architecture (ISA) Extension, used to control DMEs efficiently and with minimum latency. In particular, the firmware can use the Device API to perform these tasks:

- *Load `len` bytes from external source address to local destination address*: Use high-speed reading DMEs (identified by a particular `id`) to load data from external memory to local banked memory.
- *Store `len` bytes to external destination address from local source address*: As before, but data is stored in external memory from local banked memory with high-speed writing DMEs (also identified by a particular `id`).
- *Send `len` bytes to CGRA from local source address*: Use MM2S DMEs (identified by a particular `id`) to read data from local banked memory and send it to an input port of the CGRA.
- *Receive `len` bytes from CGRA to local destination address*: Use S2MM DMEs (identified by a particular `id`) to write data coming from an output port of the CGRA to local banked memory.
- *Route Events to `emt` emitter*: Configure the Events Switch to route the events specified by a particular bit-mask to the emitter `emt`, using a logic reduction operation, that can be OR or AND.
- *Wait for an event from an emitter*: Configure the Events Switch to route the events specified by a particular bit-mask to the emitter connected to the CPU and wait for the arrival.

3.3. Memory subsystem

At the heart of the CGR-AI Engine lies the memory subsystem, a critical component that directly influences the performance, efficiency, and scalability of the platform.

The memory subsystem in AI hardware accelerators is tasked with managing the vast amounts of data that NN models require, including weights, activations, and intermediate computations. Unlike traditional computing workloads, AI tasks often involve large datasets and require frequent memory accesses, creating unique challenges that require specialized solutions.

To address these challenges, the design of memory subsystems in AI hardware accelerators, and also for the CGR-AI Engine, must meet several key requirements:

- **High Throughput**: The memory subsystem must deliver high data throughput to match the processing speed of the processing cores, ensuring that data movement does not become a performance bottleneck.
- **Low Latency**: To support real-time applications, the memory subsystem must minimize latency, enabling rapid access to data and timely processing.
- **Energy Efficiency**: Reducing the energy cost of memory accesses is crucial and necessitates efficient data movement strategies.
- **Advanced Data Management**: Efficient data management techniques, such as intelligent caching, prefetching, and data compression, are required to optimize memory usage and access patterns, thereby enhancing overall system performance.
- **Scalability and Flexibility**: The memory subsystem must be scalable to support the growing demands of larger and more complex AI models, while also being flexible enough to adapt to different types of AI workloads.

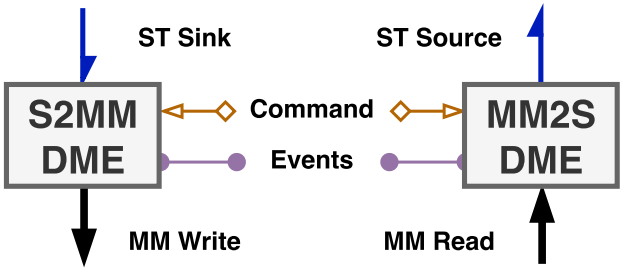


Fig. 3. DMEs handle data on streaming (blue arrows) and memory-mapped (black arrows) interfaces, receive memory transaction info on command (orange arrows) interface, and send or receive events (purple lines) through the event network. (For interpretation of the references to color in this figure legend, the reader is referred to the web version of this article.)

To meet most of these requirements, the CGR-AI Engine is equipped with a versatile multiport banked local memory whose data is moved using the DMEs controlled by the RISC-V CPU. The highest throughput (memory bandwidth of the host interface) can be achieved by ensuring that multiple accesses are served in a fixed amount of time. The data width of all elements connected to the local memory is appropriately sized to meet this requirement. Once data is brought into the local memory, DMEs employ a few cycles to issue memory read requests and feed the CGRA matrix for processing with low latency. An accurate scheduling of the memory transactions can increase the energy efficiency of the data transactions. By exploiting hardware address generators for complex access patterns, DMEs can implement advanced data management solutions by combining software approaches and hardware capabilities. The firmware running on the RISC-V is responsible for emitting all data movement commands to the DMEs. Being software programmable and highly parameterized, the banked local memory and DMEs ensure a high level of versatility and scalability.

The connection between the custom RISC-V CPU, the DMEs, and the Events Switch forms what we refer to as *SmartDMA* [47]. This module functions as a programmable Direct Memory Access Controller (DMAC), serving as a memory transaction scheduler. Kernels executed on the CGR-AI Engine implement the DFGs of specific algorithms by defining a series of data movement commands in a Data Dependency Graph (DDG). The RISC-V firmware is responsible for maintaining the DME command First In First Outs (FIFOs) filled with commands while ensuring data dependencies are respected through proper configuration of the Events Switch.

Data mover engines. MM2S and S2MM DMEs are provided with four different standardized interfaces, as shown in Fig. 3. Blue arrows represent standard Streaming (ST) interfaces, consisting of data, valid, and ready signals. These interfaces connect either to the CGRA input/output ports or link sequentially to form a MM DME. Additionally, standard MM read and write interfaces are provided, alongside two specialized interfaces for efficient transaction scheduling. Orange arrows in Fig. 3 indicate commands from the RISC-V CPU to manage data movement, while purple bidirectional connections enable event exchange with the CGR-AI Engine's central Events Switch.

The RISC-V CPU supports various memory transaction patterns through its command interface, including 1D, 2D, and 3D transactions. To initiate these, the firmware must define specific fields, including the source/destination address (`addr`), the number of bytes per row to transfer (`len`), and whether the operation begins immediately or waits for an external trigger (`go`). For more complex patterns, such as 2D and 3D transactions, additional parameters define the dimensions of the data, including the size in bytes of each row or plane (`size1d`, `size1d`) and the total number of rows or planes to transfer (`len2d`, `len3d`). In MM2S scenarios, a stream can be configured as last-terminated or left open to enable chaining multiple transactions.

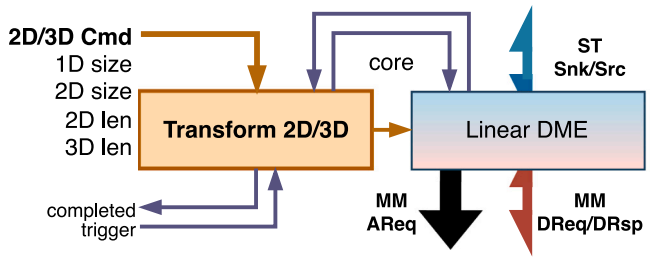


Fig. 4. 2D/3D enabled DMEs have a transformation module that convert 2D/3D commands into 1D for linear DMEs.

The `size1d` and `size2d` fields determine the type of data transfer: a non-zero `size1d` enables 2D transactions, while a non-zero `size2d` activates 3D transactions. All 2D/3D-capable DMEs include a command transformation module, as depicted in Fig. 4. This module translates 2D or 3D commands into a sequence of 1D commands tailored for the linear MM2S or S2MM DME. In the generated commands, the `last` bit remains de-asserted to chain the 1D data streams, except in the final stream, where it mirrors the state of the original 2D/3D command.

Fig. 5 shows that each DME includes two FSMs and two stream conditioning blocks for width conversion and alignment. The AREQ FSM handles DME commands and issues MM address requests when triggered, accepting new commands after each burst transaction. In MM2S, the DRSP FSM generates strobe signals based on alignment and length; in S2MM, the DREQ FSM sets the `last` signal to terminate the stream properly. The data width converter adjusts stream size, updating strobe and `last` signals. Finally, the alignment module removes or adds no-strobe bytes to ensure proper word alignment in MM2S and S2MM, respectively.

The RISC-V Control and Status Register (CSR) space has been extended to provide additional information within the DME command interface. In particular, 2D/3D memory transactions are defined by configuring dedicated CSR registers. For each channel, the list of extended CSRs includes one register per 2D/3D-related field of the DME command interface, namely `size1d`, `size2d`, `len2d`, and `len3d`. An additional CSR register is implemented for the auto-increment of the address. This solution reduces the price in clock cycles to schedule consecutive DME commands without employing an additional `add` instruction.

3.4. Events switch

In the CGR-AI Engine platform, the event network maps the `completed` output signal and the `trigger` input signal of DMEs to the event sources and emitters of the Event Switch, which is used to enforce dependencies between different memory transactions scheduled by the kernel. In addition to DMEs events, the switch is also connected to a dedicated event interface of the custom RISC-V CPU. In this way, the firmware can fire custom events and wait for external ones from DMEs. As illustrated in Fig. 6, the event sources are mapped to event emitters through a configurable switch matrix. All events are captured and cleared upon their firing condition, which can be defined either as an OR-ed or AND-ed mask. The event routing command interface is used to configure which event sources contribute to trigger each event emitter on the Events Switch. For each emitter, one register captures the event, and another saves the bit mask.

3.5. The scheduling core: RISC-V CPU

The RISC-V CPU is responsible for the orchestration of all data movements and the CGRA configuration. It exploits the CFU within the CPU of the NeoRV32 Processor [48] to implement the custom *Xsdma* ISA extension, already presented in [47]. It allows controlling MM2S and S2MM DMEs, the event switch, and CDME.

Table 2
R3-type instructions listing of the Xsdma extension.

Instr.	Fields		Registers		
	func7	func3	rs1	rs2	rd
`ev_mask`	0 000 000	000	word	mask	–
`ev_set`	0 000 000	001	id	logic	–
`ev_trig`	0 000 000	000	id	–	err?
`ev_check`	0 000 000	000	id	–	err?
`ker_load_param`	0 000 001	000	id	–	–
`info_dme_count`	1 000 000	000	–	–	–
`info_ev_emt_count`	1 000 000	001	–	–	–
`info_ev_rcv_count`	1 000 000	010	–	–	–
`info_plat_param`	1 000 000	100	id	–	–

Table 3
R4-type instructions listing of the Xsdma extension.

Instr.	Fields		Registers			
	func2	func3	rs1	rs2	rs3	rd
`dme_cmd`	00	000	addr	len	ch	new addr
`..._fire`	00	–1	addr	len	ch	new addr
`..._go`	00	-1-	addr	len	ch	new addr
`..._last`	00	1–	addr	len	ch	new addr
`dme_ch_ready`	10	100	–	–	ch	not full
`dme_ch_feat`	10	111	–	–	ch	features

3.5.1. Xsdma: Custom RISC-V extension

The DMEs are controlled by the customized RISC-V CPU using dedicated instructions. In particular, the implementation comprises custom R3- and R4-type instructions and an extended CSR space to increase configurability.

The list of custom R3-type instructions is described in Table 2. *Event mask* (`ev_mask`) and *event set* (`ev_set`) respectively prepare the mask (composed of multiple words) to select the triggering event sources and set it for the event emitter `id` using AND or OR trigger `logic`. *Event trigger* (`ev_trig`) and *event check* (`ev_check`) respectively fire an event from CPU and stall the CPU until the requested event is received. *Kernel parameter load* (`ker_load_param`) loads the requested parameter from the KMU. *Info DMEs count* (`info_dme_count`) and *Info Event Emitter and Receiver count* (`info_ev_ [emt|rcv] _count`) return respectively the number of available DME channels, event emitter ports, and event receiver ports. *Info Platform Parameter* (`info_plat_param`) returns the requested platform parameter.

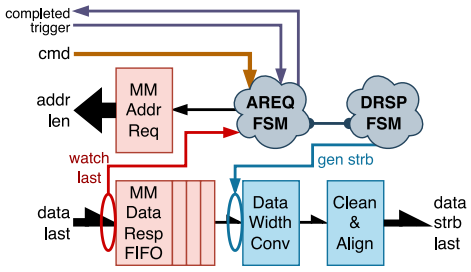
The list of custom R4-type instructions is described in Table 3. They operate with a DME distinguished by the `ch` id. *DME Schedule Command* (`dme_cmd`) sends a DME command with requested address and length. The `func3` field is used as bit-mask to enable three properties of the scheduled command:

- *Fire completed event* (`_fire`): If set, when the command is completed, DME will fire an event.
- *Go without trigger* (`_go`): If set, the DME will not wait for a trigger.
- *Last termination* (`_last`): If set, the data stream generated by the MM2S DMEs will be terminated by asserting the last bit.

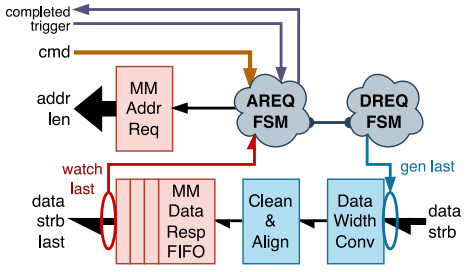
DME channel ready (`dme_ch_ready`) returns true if the requested DME channel can receive new commands. Finally, *DME channel features* (`dme_ch_feat`) returns a bit mask used to determine the DME capabilities: MM2S or S2MM, with 2D or 3D data movement enabled.

The implemented *Xsdma* takes advantage of the readable and writable CSRs of RISC-V ISA that are reserved for custom use to implement an extended address space, as described in Table 4. A CSR read operation on `xcsr_data` returns the content of the extended CSR addressed by `xcsr_addr`, similarly, a write operation updates the content of the extended CSR.

This space provides additional information within the DME command interface. In particular, 2D/3D memory transactions are defined



(a) MM2S DME watches for the last bit assertion on the MM Data Response interface, and generates the correct strobe bits of the stream.



(b) S2MM DME generates the last bit on the data stream based on the length, and watches for the last bit assertion on the MM Data Request interface.

Fig. 5. DMEs interface the memory with Address Request (AREQ) and Data Response/Request (DRSP/DREQ) FSMs, and process the data stream using width conversion and alignment modules.

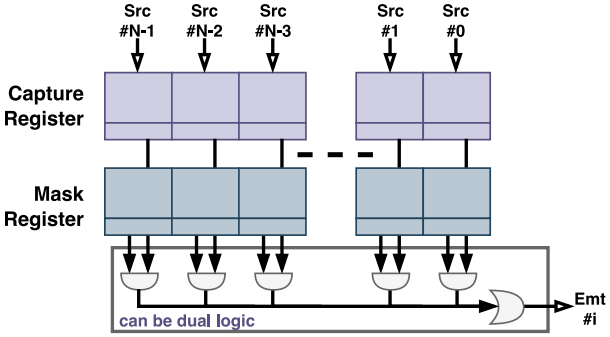


Fig. 6. Diagram of a single event emitter logic in the Events Switch. Events are captured in a slice of registers and firing logic is applied to masked events.

Table 4
Custom user-mode R/W CSRs of the RISC-V ISA.

CSR	Name	Description
0x800	`xcsr_addr`	Address of the register in the extended space
0x801	`xcsr_data`	Data to read/write in the extended register

by configuring dedicated CSRs. The address of the extended space comprises five bits to select the DME channel and four bits to select the register. For each channel, the list of extended CSRs includes one register per 2D/3D-related field of the DME command interface, namely `size1d`, `size2d`, `len2d`, and `len3d`.

4. Case study: Convolutional Neural Network inference

Mapping NN layers onto the CGR-AI Engine requires a multi-step process to efficiently allocate each layer's computations across the architecture's available resources. This task is critical for optimizing performance, energy efficiency, and parallelism during NN inference. The challenge lies in the diverse operations of NNs—such as convolutions, pooling, matrix multiplications, and activations—that must be strategically distributed across the PEs within the CGRA core.

The mapping process consists of the following key steps:

- **Model Layer Decomposition**: The first step is to decompose the NN into its constituent layers. Each layer (e.g., convolutional, fully connected, pooling) has distinct computational patterns and data dependencies. Decomposing these layers helps to understand how each operation can be parallelized and mapped onto the CGRA rows. For example, a convolutional layer involves many matrix multiplications, which can be distributed between the CGRA rows in a parallel manner.
- **Operation Scheduling**: Once the layers are decomposed, the next step is to determine the schedule to execute the operations. This involves deciding the order in which operations should be assigned to PEs, ensuring that data dependencies are respected.
- **PE Allocation and Placement**: After scheduling, the actual placement of the operations in PEs is determined. Each PE can execute certain types of operations (e.g., multiplication, addition), and the placement algorithm assigns operations from the NN layers to the appropriate PEs based on their capabilities. For example, in a convolutional layer, different portions of the convolution operation can be mapped to different PEs, implementing MAC operations or activation functions.
- **Dataflow and Routing**: Performing data routing efficiently between PEs is a crucial step in CGRA mapping. Data generated by one PE may need to be routed to another PE for further processing. Routing mechanisms are optimized to minimize communication overhead and maximize data reuse within the local memory of the device, reducing costly external memory accesses. Data flow optimization ensures that data movement is synchronized with computation to avoid bottlenecks.
- **Resource Constraints and Optimization**: The mapping process must account for resource constraints, such as the number of

CGRA rows, the available local memory, and the bandwidth. Optimizing for these constraints is critical to achieving high performance and low power consumption. Techniques such as loop unrolling, tiling, and data reuse are commonly employed to ensure that resources are used efficiently [49].

- **Execution and Reconfiguration**: Once the NN layer has been mapped onto the CGRA, the kernel schedules the memory transaction related to the assigned operations in parallel among the CGRA rows. The CGRA core can be dynamically reconfigured for different layers or portions of the network, adapting to the changing computational requirements of each layer. This reconfigurability allows the CGR-AI Engine to handle different layers with varying computational characteristics efficiently.

The following subsections describe how these steps can be implemented for each CNN layer.

4.1. Kernel structure

To assess the functionalities of the CGR-AI Engine platform, several kernels for the NN layers have been developed and are described in the following sections. Each kernel is parameterized and designed to allow exploring different configurations of the CGR-AI Engine, varying the available CGRA rows, the precision of PEs, the number of high-speed MM DMEs, and the available local memory. Each kernel is compiled using the bare metal GCC compiler for RISC-V, with *newlib* standard C library. The structure of a kernel is the following:

1. Define and load the kernel parameters.
2. Configure the CGRA matrix.
3. Setup DMEs CSRs.
4. Ensure data dependency by configuring the Event Switch.
5. Schedule MM DME commands to read data in local memory.
6. Schedule CGRA DME commands to process data with CGRA.
7. Schedule MM DME commands to write data back to external memory.

Each kernel defines its set of parameters that can be loaded from KMU with a dedicated instruction. In particular, the host can set these parameters to test different sizes of the algorithms. Once started, the kernel loads all the parameters and calculates additional constants that are used later by the scheduling algorithm.

Kernels can define the CGRA configurations at run-time or can use pre-generated ones that are statically included in the binary. The CGRA is reconfigurable at run-time by providing a configuration word that specifies, for example, what FU output each FU should use as input. This can be done directly in the C code by invoking dedicated driver functions to configure the CGRA or by including pre-defined configuration data within the compiled binary code and referencing it at run time.

The same considerations apply to the routing of the CGRA. Configurations can be modified at runtime to map DFGs on different Tiles and connect them accordingly; otherwise, statically provided configurations can be directly loaded on CGRA.

Data must be loaded from external memory before processing with the CGRA core. To do so, DME commands are scheduled on High-Speed Reading MM DMEs. The Event Switch is configured to route the completed event of a S2MM DME (the one writing in local memory) to the CGRA MM2S DMEs (the ones feeding CGRA). In this way, processing is ensured to start only when data is loaded into the local memory.

To execute the DFG, the data must be loaded from local memory using the CGRA MM2S DMEs. Depending on the algorithm, the kind of memory transaction can be 1D, 2D, or 3D. The first data moving commands have to wait for the trigger arriving from High-Speed Reading S2MM DMEs. The last command will fire the completed event to synchronize the write-back procedure.

To write-back data to the external memory, the Event Switch must be configured to trigger High-Speed Writing MM DMEs when the CGRA S2MM DMEs complete their transactions. Lastly, a S2MM DME (the one that writes to the external memory) will fire the completed event to signal the termination of the kernel. As anticipated, the Event Switch is configured to route the completed event of the write-back procedure to the RISC-V CPU. With a dedicated instruction, the kernel waits for its arrival before terminating its execution.

4.2. Mapping of convolutional neural network layers

Starting from the DFG representation of a CNN, each operation in every layer is manually mapped onto the CGRA fabric. Since no compiler targeting the specific architectural constraints of our platform is currently available, the mapping process is performed statically. Parameters such as the set of FUs implemented in each tile are defined at design time. In contrast, others, such as the routing and propagation of data between FUs, are configured at run time via dedicated software drivers. In the following, we describe how the different layers of a CNN can be statically mapped onto the CGRA architecture.

Dense layers. Dense layers have few parameters, and their size depends on the input feature map. Kernel parameters are input address and sizes, filter address and channels, and output address. To clarify how each of these steps is executed on the proposed platform, we provide pseudo-C code examples specifically for this type of layer:

```
typedef struct kernel_params {
  // Direct parameters
  uint8_t  *input;
  uint32_t input_w;
  uint32_t input_h;
  uint32_t input_ch;
  uint8_t  *filter;
  uint32_t filter_ch;
  uint8_t  *output;

  ...
```

From the direct parameters, the following derived constants are calculated:

```
  ...

  // Derived parameters
  uint32_t input_size;
  uint32_t filter_size;
  uint32_t filters_size;
  uint32_t output_size;
  uint8_t *input_laddr;
  uint8_t *filter_laddr;
  uint8_t *output_laddr;
} kernel_params_t;

int main(void)
{
  params.input_size   = params.input_h * params.
      input_w * params.input_ch;
  params.filter_size  = params.input_size;
  params.filters_size = params.filter_size *
      params.filter_ch;
  params.output_size  = params.filter_ch *
      platform.vector_size;

  // Local addresses are statically chosen
      starting from
  // symbol defined in the linker script
  params.input_laddr  = (&__cgrai_data_start);
  params.filter_laddr = params.input_laddr +
      params.input_size;
```

```
  params.output_laddr = params.filter_laddr +
      params.filters_size;

  ...
```

To compute a fully connected layer, only MAC operations are needed. Therefore, the configuration of the CGRA comprises a MAC unit on the first column, and a forward operation on the second column:

```
  ...

// MAC

  // FU Mul uses the first and second PE inputs
  cfg_1.pe.fus.mul.incf.src[0].sel =
      CGRA_FU_IN_PE;
  cfg_1.pe.fus.mul.incf.src[1].sel =
      CGRA_FU_IN_PE + 1;
  // Signed multiplication
  cfg_1.pe.fus.mul.opcode = CGRA_FU_MUL_OP_MUL;

  // FU Add uses only Mul output
  cfg_1.pe.fus.add.incf.src[0].sel =
      CGRA_FU_IN_MUL;
  cfg_1.pe.fus.add.incf.src[1].sel =
      CGRA_FU_IN_ZERO;
  // Signed ADD with accumulation
  cfg_1.pe.fus.add.opcode = CGRA_FU_ADD_OP_ADD;
  cfg_1.pe.fus.add.accum  = 1;

  // PE output is FU Add output
  cfg_1.pe.out_sel[0] = CGRA_PE_OUT_ADD;

// Forward

  // PE on the second column forwards its first
      input
  cfg_2.pe.out_sel[0] = CGRA_PE_OUT_IN;

  ...
```

Both MM2S and S2MM DMEs perform a 1D data movement and exploit the auto-increment feature to save clock cycles in the scheduling algorithm:

```
  // Input DMEs
  for (int i = 0; i < platform.dme_mm2s_count; i
      ++) {
    int ch = platform.dme_mm2s_start_ch + i;

  // Auto-increment only the second DME to span
      filters
  if (i     cfu_csr_dmech_write(ch,
      CFU_CSR_DMECH_INCR, params.filter_size);
  }

  // Output DMEs
  for (int i = 0; i < platform.dme_s2mm_count; i
      ++) {
    int ch = platform.dme_s2mm_start_ch + i;

    // Increment output by a vector size (in
        bytes)
    cfu_csr_dmech_write(ch, CFU_CSR_DMECH_INCR,
        platform.vector_size);
   }
```

The scheduling algorithm spans over filters to generate all outputs:

```
  uint8_t * output_addr = params.output_laddr;
  uint8_t * input_addr  = params.input_laddr;
  uint8_t * filter_addr = params.filter_laddr;

  // For each output channel...
  for (int ch = 0; ch < params.filter_ch; ch +=
      platform.rows) {

    // Span over CGRA rows
    for (int row = 0; row < platform.rows; row
        ++) {
      if (ch + row < params.filter_ch) {

        // If these are last commands on each
            row, schedule firing commands
        if (ch >= params.filter_ch - platform.
            rows)
          output_addr = cfu_dme_cmd_fire_go_last
              (
                          output_addr,
                          platform.vector_size -
                              1,
                          row + platform.
                              dme_s2mm_start_ch)
                              ;

        // Otherwise, schedule silent commands
        else
          output_addr = cfu_dme_cmd_go_last(
                          output_addr,
                          platform.vector_size -
                              1,
                          row + platform.
                              dme_s2mm_start_ch)
                              ;

        // First iteration waits for trigger
            from High-Speed S2MM DME
        if (ch == 0) {
          cfu_dme_cmd_last(
            input_addr,
            params.filter_size - 1,
            2 * row + platform.dme_mm2s_start_ch
                );

          filter_addr = cfu_dme_cmd_last(
                          filter_addr,
                          params.filter_size -
                              1,
                          2 * row + platform.
                              dme_mm2s_start_ch
                              + 1);

        // Successive iterations can "go"
        } else {
          cfu_dme_cmd_go_last(
            input_addr,
            params.filter_size - 1,
            2 * row + platform.dme_mm2s_start_ch
                );

          filter_addr = cfu_dme_cmd_go_last(
                          filter_addr,
                          params.filter_size -
                              1,
                          2 * row + platform.
                              dme_mm2s_start_ch
                              + 1);
        }
      }
    }
  }
```

To keep the description concise, we refrain from presenting the full pseudo-code for the remaining CNN layers. In any case, by writing

the C code, it is possible to configure the CGRA core with a setup suitable for the execution of multiple layers. The user is responsible for developing the firmware to orchestrate data movement between global and local memory, as well as between local memory and the CGRA, across different layers.

Convolutional layers. 2D Convolutional layers have many parameters to customize them: input address and sizes, filter address and sizes, stride values, and output address. To compute a Conv2D layer, only MAC operations are needed. Therefore, the same CGRA configurations of dense layers are used. While S2MM DMEs perform a 1D data movement, MM2S ones need to read data in a 3D space and the related CSRs must be set. The auto-increment feature is exploited to save clock cycles in the scheduling algorithm by incrementing filter and output addresses. Again, the scheduling algorithm spans over output channels to perform the convolution.

Pooling layers. Pooling layers do not come with a set of filters. The size of the sliding window can be customized, and the input data is processed directly. In particular, the parameters are input address and sizes, pool window sizes, stride values, and output address.

- **Average Pooling** is computed with only an accumulation operation. The division factor can be embedded in the filter values of the next layer through training. Therefore, the configuration of the CGRA comprises an ADD FU on the first column and a bypass operation on the second column.
- **Max Pooling** is computed with only a comparison operation on the data stream. Therefore, the configuration of the CGRA once again comprises an ADD FU on the first column and a bypass operation on the second column.

Although S2MM DMEs carry out a one-dimensional data transfer, MM2S DMEs are required to access data within a two-dimensional window on each input channel, requiring the configuration of associated CSRs. Once again, the auto-increment feature is used to save clock cycles in the scheduling algorithm by incrementing both input and output addresses. The scheduling algorithm of pooling layers spans over output channels in the same way as the Conv2D layer.

Activation functions. Activation functions generally require minimal parameterization. Common parameters include the input address, input size, and output address. Additional parameters may be necessary depending on the specific function. For example, the kernel might need the LUT data address to initialize the CGRA LUT FU, which is used to store quantized values of nonlinear functions. All activation functions, except for the SoftMax kernel, also include a `batch_size` parameter, which facilitates splitting the data loading process into smaller batches for distribution across multiple CGRA rows.

- **ReLU** involves a simple comparison operation with zero on a single data stream. Its CGRA configuration consists of an ADD FU in the first or second column, with a bypass operation on the other column. This operation can be merged with the Conv2D kernel since it occupies only one Tile per row.
- **ReLU6** uses two comparison operations, one with zero and another with the threshold value six. The CGRA configuration includes two ADD FUs, one per column. Unlike ReLU, this operation cannot be merged with the Conv2D kernel because it requires an additional ADD FU for accumulation.
- **LeakyReLU** combines a signed multiplication and a comparison with zero. Its CGRA configuration includes one MUL FU and one ADD FU. Similar to ReLU, it can be merged with the Conv2D kernel as it occupies only one Tile per row.
- **LUT Loading** is performed prior to executing the Exponential Linear Unit (ELU) and SoftMax kernels. This step involves loading the approximated exponential function into the LUT of Tiles in the second column of the CGRA core. The process follows the standard procedure for loading 1D data from external memory to CGRA rows. So far, the LUT is 5-bit addressable, allowing up to 32 quantized values to be stored simultaneously. If additional values are required, multiple LUT loading operations can be scheduled accordingly. Since this work primarily focuses on performance metrics, accuracy values for the executed non-linear functions are not reported.
- **ELU** is computed using the LUT memory within the core and a comparison with zero. Its CGRA configuration comprises an ADD FU and a LUT FU in the second column, with a bypass operation in the first. Like ReLU, this operation can be merged with the Conv2D kernel due to its minimal resource usage.
- **SoftMax** utilizes a LUT for the exponential function and an accumulator for computing the division factor. The CGRA configuration includes an ADD FU and a LUT FU on the second column, with a forward operation on the first. Although it occupies only one Tile per row and could theoretically be merged with the Conv2D kernel, the division step must be performed externally—by the CPU or host—due to the lack of a division unit in the CGRA.

5. Engine evaluation and characterization

Following the same approach as for the SmartDMA evaluation in [47], the demonstration System on Chip (SoC) illustrated in Fig. 7 is set up with five configurations selected for the DSE activity, which are described in Table 5. Such bit widths have been chosen because quantized NNs run on low-precision arithmetic. The other parameters have been designed to sustain the available bandwidth in high-speed ports. The number of banks, the vector size and the local memory data width are set to allow CGRA rows to access the memory simultaneously and keep the CGRA input FIFOs full. It is worth noting that the architecture does not include a NoC. Instead, the interconnections of an Advanced eXtensible Interface (AXI) bus are simulated during synthesis, under the assumption that the required bandwidth can be reached by the interface.

5.1. Standard cell characterization

For each configuration mentioned, a comprehensive Performance, Power, and Area (PPA) analysis was carried out by synthesizing the CGR-AI Engine using the DARE65T RHBD standard cell library platform developed by imec. The Design Against Radiation Effects (DARE) platform is based on a commercial 65 nm technology process, providing both digital and analog Intellectual Property (IP) cells, specifically designed to mitigate radiation effects. In particular, the digital core library features three threshold-voltage options, operates at 1.2 V, and is qualified up to a Total Ionizing Dose (TID) of 100 krad with Single Event Latch-ups (SELs) occurring at a Linear Energy Transfer (LET) of $70\,\mathrm{MeVcm^2/mg}$ [50].

We performed the synthesis under varying constraints for the desired clock frequency, ranging from 100 MHz to 333 MHz, which represents the maximum frequency at which timing constraints are satisfied. TCM and banked local memories are treated as macros and excluded from the total cell area count. In particular, we employed the automation framework proposed in [51], referred to as AutoDEV, to orchestrate the synthesis process using Synopsys Design Compiler 2022.12, and time-based power analysis with Synopsys PrimePower 2022.12.

From the performed synthesis, we observe, as expected, that both area occupation and power consumption increase with the target frequency. To meet the more stringent timing constraints imposed by higher frequencies, the synthesis tool selects standard cells with stronger drive strengths, which inherently occupy more area. Additionally, techniques such as logic retiming and the insertion of buffers

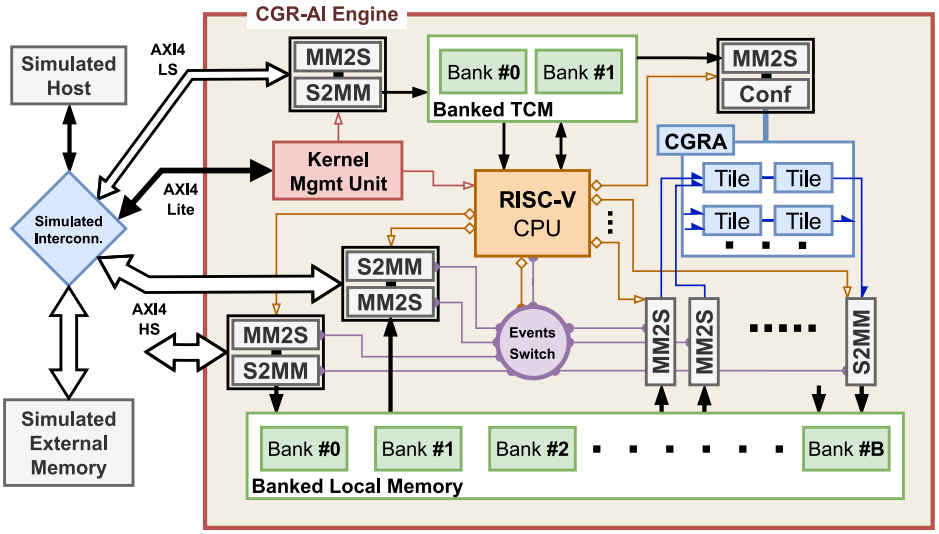


Fig. 7. CGR-AI Engine IP connected to a simulated host system composed of CPU, AXI interconnect, and external memory.

Table 5
Summary of CGR-AI Engine configurations. Bit-widths in Local Memory and High-Speed Ports indicate the width of the data bus. R and W in High-Speed Ports indicate the number of reading and writing ports.

Config. name	Element size	Vector size	CGRA rows	Local memory (banks)	# High-Speed ports
A	8-bit	32-bit	8	512kB (8-banks, 256-bit)	2R/2W (512-bit)
B	8-bit	64-bit	3	64kB (4-banks, 128-bit)	1R/1W (512-bit)
C	8-bit	64-bit	6	256kB (4-banks, 128-bit)	2R/2W (256-bit)
D	8-bit	128-bit	1	64kB (2-banks, 256-bit)	1R/1W (512-bit)
E	16-bit	64-bit	3	64kB (4-banks, 128-bit)	1R/1W (512-bit)

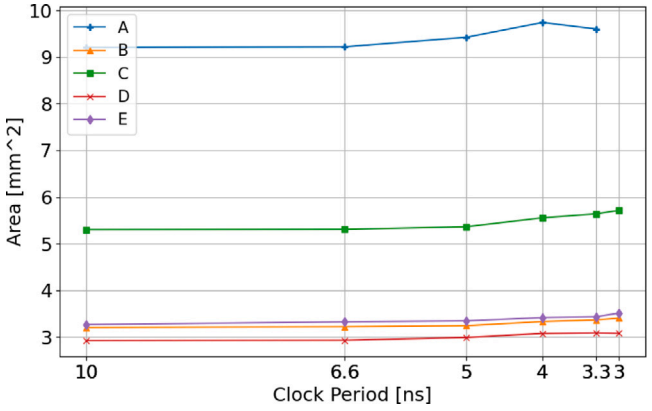


Fig. 8. Area occupation for each CGR-AI configuration implemented on the DARE65T library platform.

further contribute to the area increase. Fig. 8 illustrates how the area of each CGR-AI configuration scales with increasing frequency. As for power, we do not include a dedicated plot, since its growth aligns with the expected trend, following the power dissipation formula. It is also interesting to note that the critical path is typically found within the multiplier FU.

As expected, the area increases with the number of rows, and consequently, the number of tiles implemented in the design. It is important to note that TCMs and local memories are excluded from the area count, as they are synthesized as macros. Therefore, the area differences across configurations are primarily driven by the vectorization level and the number of rows. For instance, configurations B and E exhibit a similar area trend, as they share the same vectorization level and number of rows. Interestingly, configuration D also follows a similar trend, despite having three times fewer tiles. This is largely due to its higher vectorization level, with a data width of 128 bits, which leads to increased area occupation. We refrain from providing a detailed breakdown of the CGRA core and SmartDMA area, as this information is already available in previous works [44,45,47].

Our analysis focuses on area efficiency and energy efficiency, which are quantified as operations per second per unit area (GOps/mm^2) and operations per second per unit power (GOps/W = GOp/mJ). The area and energy efficiencies of the synthesized configurations of the CGR-AI Engine are calculated and listed in Table 6. Since the area slightly increases with frequency, the best area efficiency is found at maximum frequency, while the best energy efficiency is found at 150 MHz, even if the value at 333 MHz is still acceptable (decreased only by 4%–6%).

We refrain from providing quantitative comparisons with other SotA works, as such comparisons would be challenging and potentially misleading. Many existing solutions either focus solely on the CGRA core or present results within dedicated frameworks and benchmarking environments. Since a comparison at the core level has already been presented in [44], we do not extend this analysis to the CGR-AI Engine, whose heterogeneous nature makes direct comparison even less straightforward.

5.2. Benchmarking results on CNN layers

Each kernel previously presented is executed to obtain execution latency values. The benchmark setup is reported in Table 7. The kernels have been run with different parameters to understand the best practices for CGRA resource utilization and the bottlenecks of the scheduling process. In particular, the kernels are executed for each of the five configurations of the Engine at a 300 MHz frequency. The number of rows determines the degree of parallelism in the convolutional layers, as each row contains a tile equipped with a MAC unit

Table 6
Throughput [GOps] (a), area efficiency [GOps/mm^2] (b), and energy efficiency [GOps/W] (c) of synthesized CGR-AI configurations. In bold, are highlighted the best metrics.

Name	DW	Rows	Vect	150 MHz			200 MHz			250 MHz			300 MHz			333 MHz		
				(a)	(b)	(c)	(a)	(b)	(c)	(a)	(b)	(c)	(a)	(b)	(c)	(a)	(b)	(c)
A	32	8	8	4.85	0.526	**18.7**	6.40	0.679	18.5	8.00	0.821	18.4	9.70	**1.01**	18.2	–	–	–
B	64	3	8	3.64	1.13	**37.9**	4.80	1.48	37.3	6.00	1.8	36.8	7.27	2.16	36.2	8.00	**2.35**	35.7
C	64	6	8	7.27	1.37	**40.8**	9.60	1.79	40.1	12.00	2.16	39.6	14.55	2.58	38.9	16.00	**2.8**	38.3
D	128	1	8	2.42	0.826	**31.1**	3.20	1.07	30.6	4.00	1.3	30.6	4.85	1.57	29.8	5.33	**1.73**	29.6
E	64	3	16	1.82	0.547	**18**	2.40	0.717	17.8	3.00	0.878	17.8	3.64	1.06	17.7	4.00	**1.14**	17.3

Table 7
List of conducted benchmarks with parameters.

Kernel	Parameters Sweep
Dense	**Size**: 256, 16384, 65536; **Channels**: 128, 1000, 4096;
Conv2D	**Size**: 3 × 3 × 256 in 64 × 64 × 3, 3 × 3 × 256 in 128 × 128 × 3, 5 × 5 × 512 in 256 × 256 × 10; **Stride**: 1, 2, 3;
Avg/Max Pool	*Same as Conv2D*
[Leaky]ReLU [6]	**Size**: 128, 512, 2048; **Batches**: 32, 64, 128;
ELU/SoftMax	*Same as ReLUs*
Conv2D + ReLU	*Same as Conv2D*
Conv2D + ELU	*Same as Conv2D*

responsible for performing the convolution operation. Execution latencies have been measured with post-synthesis functional simulations using QuestaSim 22.4.

For each CGR-AI Engine configuration, the kernels are executed and the latency is measured from the first scheduled command on the high-speed MM reading DME to the last ack on the high-speed MM port. Fig. 9 illustrates the dependency of kernel latency primarily on the size of the input data. The results exclude latency on the external memory interface, assuming zero delays in serving requests and no bus contentions.

Notably, the most time-consuming kernels are also the most compute-intensive, particularly *Conv2D* and *Dense*, due to their extensive MAC operations. Among the five configurations analyzed, the last configuration demands the highest execution time, whereas the others demonstrate relatively balanced performance. Nevertheless, while execution times for the same kernel across different configurations may appear very similar, it is important to note that we aimed to achieve comparable throughput across all of them to evaluate the impact of architectural differences better. Additionally, since the graph depicts clock cycles, we can observe from the graphs that the architecture exhibits deterministic behavior as expected. Consequently, implementing the engine at higher frequencies reduces latency.

Based on these results, we can derive a rough estimation of the execution time for a full CNN. For instance, MobileNet-V1 [52] consists of 28 convolutional layers (mostly 3 × 3 with stride 1), one pooling layer (7 × 7 with stride 1), and a fully connected layer with 1000 neurons, operating on an input size of 224 × 224 × 3. Although the dimensions of the benchmarks do not always align with the network's layers, we can still derive an estimation by making reasonable assumptions. For example, considering the 8-row configuration, with up to 8 tiles working in parallel, let us assume that the 28 convolutional layers each have an execution time matching that of *conv2D + ReLU* for the second kernel size ($\sim 6 \times 10^4$ cycles).

Furthermore, we assume that the pooling layer has an execution time comparable to the 5 × 5 kernel ($\sim 6 \times 10^3$ cycles), and that the fully connected layer with 1000 channels requires approximately 5×10^5 cycles. Based on these assumptions, we can derive the inference time in seconds. Specifically, if we consider a 300 MHz frequency, we can calculate an approximate latency of $\sim$ 7.29 ms. It is worth noting that storing all the network parameters in local memory is not necessary, especially for a large network like MobileNet, as it can be partitioned into multiple segments whose parameters can be progressively loaded from global to local memory.

We can compare the estimated result with two other SotA solutions: Eyeriss v2 [40] and a Commercial Off The Shelf (COTS) solution like the Myriad-X [53,54]. With the Myriad, MobileNet is executed at 700 MHz with a 10 ms inference time and a power dissipation of 1500 mW, compared to the 533.3 mW of our solution. This comparison suggests that our engine has the potential to outperform advanced COTS solutions. On the other hand, Eyeriss performs inference in just 0.68 ms at 200 MHz. While our solution is theoretically outperformed in this case, the results remain promising, as they are not significantly behind one of the most advanced SotA CGRA-based solutions.

6. Conclusion

To address the growing interest in ML for space-related applications, a long-term solution for high-class missions is needed. The SotA for data processing in deep space missions does not include high-performance GPUs, which do not have radiation hardening characteristics. The other solutions, CPUs and FPGAs, do not provide enough computing power or efficiency to execute ML tasks onboard. This work tackles the problem of DSE of CGRA platforms to provide a framework and a solution for the use of such devices in space.

Specifically, we incorporated our CGRA and SmartDMA architectures into a CGR-AI Engine that represents a fully operational platform for computing NNs on the edge. DSE activity on high-level parameters has been performed to characterize the platform on DARE65T technology for CNN workloads. It shows results between 17.3 GOps/W and 40.8 GOps/W. Finally, the mapping of kernels implementing CNN layers onto the proposed platform is analyzed. The discussion focuses on the specific requirements and considerations for mapping and executing each layer onto the platform, highlighting the potential advantages of the proposed solution compared to SotA approaches in terms of performance and energy efficiency.

In conclusion, the CGR-AI Engine platform and its DSE framework represent a valid solution to bring a programmable solution for NN inference in the space domain. The results are consistent with those achievable through expensive FPGA development methodologies that are needed in the space sector. The programmability of this kind of architecture makes CGR-AI a versatile solution to bring AI into space. Considering the lack of rad-hard CGRA architectures targeting space applications in the literature, we believe that this work can serve as the basis for a CGRA-based onboard system to be used in future space missions.

Declaration of competing interest

The authors declare the following financial interests/personal relationships which may be considered as potential competing interests: Luca Zulberti reports financial support was provided by European Space Agency. Pietro Nannipieri reports financial support was provided by Italian Ministry of University and Research. If there are other authors, they declare that they have no known competing financial interests or personal relationships that could have appeared to influence the work reported in this paper.

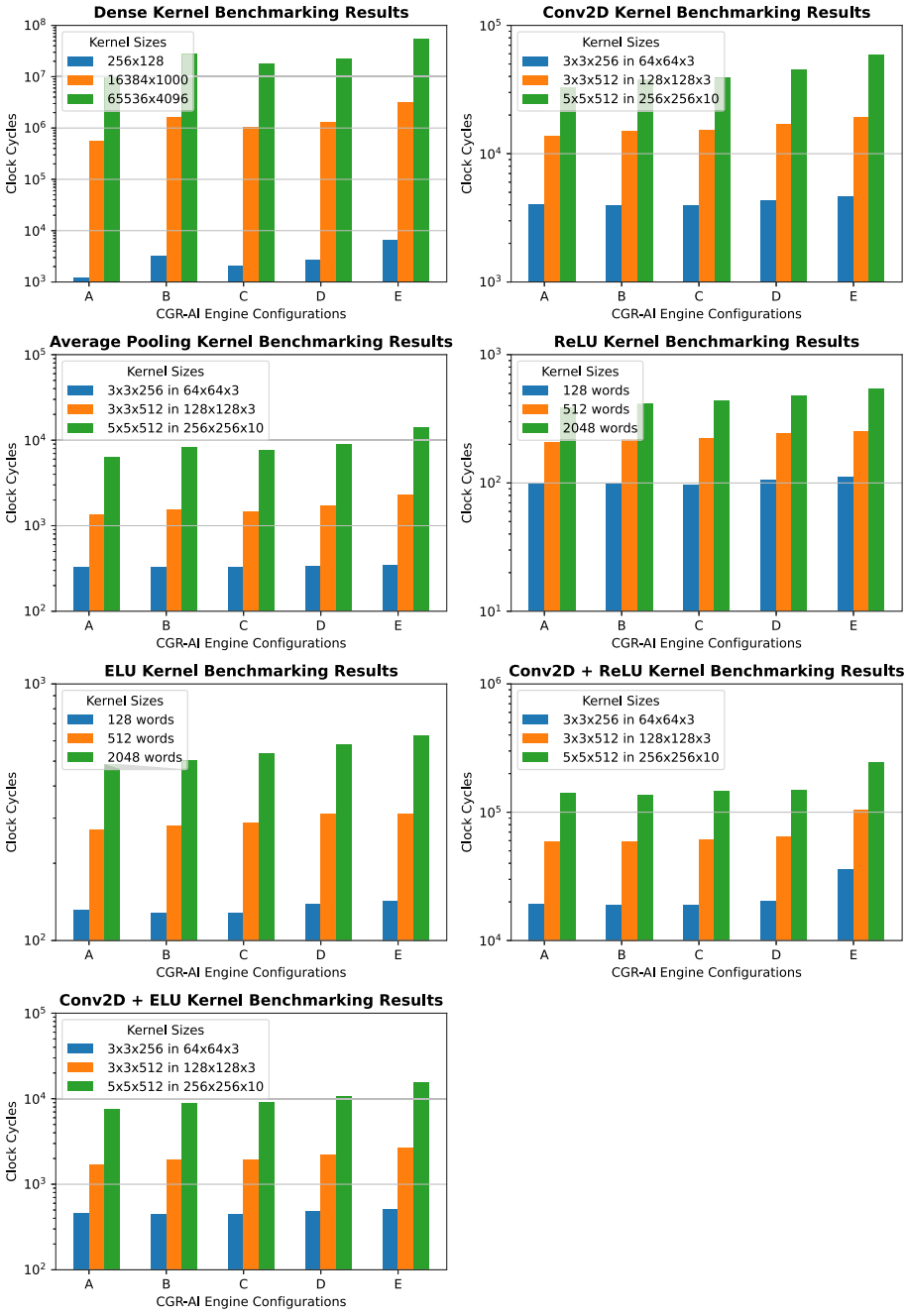


Fig. 9. Each plot shows how the kernels performed in terms of clock cycles for each CGR-AI configuration.

Acknowledgments

Funding: This study received funding from the European Union - Next-GenerationEU - National Recovery and Resilience Plan (NRRP) – MISSION 4 COMPONENT 2, INVESTMENT N. 1.1, CALL PRIN 2022 PNRR D.D. 1409 14-09-2022 – (OPERAND - a recOnfigurable Platform & framEwork foR Ai infereNce on the eDge) CUP N.I53D23006070001, from the Italian Ministry of University and Research (MUR) within the framework of the FoReLab project (Departments of Excellence) and from the European Space Agency (ESA), France within the framework of the Open Space Innovation Platform (OSIP) co-funded research under the ESA contract No. 4000137503/22/NL/GLC/MY.

Data availability

No data was used for the research described in the article.

References

[1] D. Izzo, G. Meoni, P. Gómez, D. Dold, A. Zoechbauer, Selected trends in artificial intelligence for space applications, 2022, http://dx.doi.org/10.48550/arXiv.2212.06662.

[2] G. Furano, G. Meoni, A. Dunne, D. Moloney, V. Ferlet-Cavrois, A. Tavoularis, J. Byrne, L. Buckley, M. Psarakis, K.-O. Voss, L. Fanucci, Towards the use of artificial intelligence on the edge in space systems: Challenges and opportunities, IEEE Aerosp. Electron. Syst. Mag. 35 (12) (2020) 44–56, http://dx.doi.org/10.1109/MAES.2020.3008468, URL https://ieeexplore.ieee.org/document/9288809/.

[3] L. Marquez, E. Fragkopoulou, K. Cavanaugh, H. Houskeeper, J. Assis, Artificial intelligence convolutional neural networks map giant kelp forests from satellite imagery, Sci. Rep. 12 (1) (2022) 22196, URL https://www.nature.com/articles/s41598-022-26439-w.

[4] B. Zhang, Y. Wu, B. Zhao, J. Chanussot, D. Hong, J. Yao, L. Gao, Progress and challenges in intelligent remote sensing satellite systems, IEEE J. Sel. Top. Appl. Earth Obs. Remote. Sens. 15 (2022) 1814–1822, URL https://ieeexplore.ieee.org/document/9705087.

[5] G. Furano, A. Tavoularis, M. Rovatti, AI in space: applications examples and challenges, in: 2020 IEEE International Symposium on Defect and Fault Tolerance in VLSI and Nanotechnology Systems, DFT, 2020, pp. 1–6, http://dx.doi.org/10.1109/DFT50435.2020.9250908, URL https://ieeexplore.ieee.org/document/9250908.

[6] G. Guerrisi, G. Bencivenni, G. Schiavon, F. Del Frate, On-board multispectral image compression with an artificial intelligence based algorithm, in: IGARSS 2024-2024 IEEE International Geoscience and Remote Sensing Symposium, IEEE, 2024, pp. 2555–2559, URL https://ieeexplore.ieee.org/document/10642517.

[7] G. Giuffrida, L. Fanucci, G. Meoni, M. Batic, L. Buckley, A. Dunne, C. van Dijk, M. Esposito, J. Hefele, N. Vercruyssen, G. Furano, M. Pastena, J. Aschbacher, The Φ-sat-1 mission: The first on-board deep neural network demonstrator for satellite earth observation, IEEE Trans. Geosci. Remote Sens. 60 (2022) 1–14, http://dx.doi.org/10.1109/TGRS.2021.3125567, URL https://ieeexplore.ieee.org/document/9600851/.

[8] S. Kacker, A. Meredith, K. Cahoy, G. Labreche, Machine learning image processing algorithms onboard OPS-SAT, in: Proceedings of the 36th Annual AIAA/USU Conference on Small Satellites, AIAA/USU, Logan, UT, USA, 2022, SSC22-P2-23.

[9] G. Meoni, M. Märtens, D. Derksen, K. See, T. Lightheart, A. Sécher, A. Martin, D. Rijlaarsdam, V. Fanizza, D. Izzo, The OPS-SAT case: A data-centric competition for onboard satellite image classification, Astrodynamics 8 (4) (2024) 507–528.

[10] G. Labrèche, D. Evans, D. Marszk, T. Mladenov, V. Shiradhonkar, T. Soto, V. Zelenevskiy, OPS-SAT spacecraft autonomy with TensorFlow lite, unsupervised learning, and online machine learning, in: 2022 IEEE Aerospace Conference, AERO, IEEE, 2022, pp. 1–17.

[11] G. Furano, A. Tavoularis, M. Rovatti, AI in space: Applications examples and challenges, in: 2020 IEEE International Symposium on Defect and Fault Tolerance in VLSI and Nanotechnology Systems, DFT, IEEE, 2020, pp. 1–6.

[12] G. Furano, G. Meoni, A. Dunne, D. Moloney, V. Ferlet-Cavrois, A. Tavoularis, J. Byrne, L. Buckley, M. Psarakis, K.-O. Voss, et al., Towards the use of artificial intelligence on the edge in space systems: Challenges and opportunities, IEEE Aerosp. Electron. Syst. Mag. 35 (12) (2020) 44–56.

[13] D. Rijlaarsdam, T. Hendrix, P.T.T. González, A. Velasco-Mata, L. Buckley, J.P. Miquel, O.A. Casaled, A. Dunne, The next era for earth observation spacecraft: An overview of CogniSAT-6, IEEE J. Sel. Top. Appl. Earth Obs. Remote. Sens. 18 (2025) 2450–2463, http://dx.doi.org/10.1109/JSTARS.2024.3509734.

[14] K. Simonyan, A. Zisserman, Very deep convolutional networks for large-scale image recognition, 2014, URL http://arxiv.org/abs/1409.1556.

[15] M. Verhelst, B. Moons, Embedded deep neural network processing: Algorithmic and processor techniques bring deep learning to IoT and edge devices, IEEE Solid-State Circuits Mag. 9 (4) (2017) 55–65, http://dx.doi.org/10.1109/MSSC.2017.2745818, URL https://ieeexplore.ieee.org/document/8110869.

[16] M. Amani, A. Ghorbanian, S.A. Ahmadi, M. Kakooei, A. Moghimi, S.M. Mirmazloumi, S.H.A. Moghaddam, S. Mahdavi, M. Ghahremanloo, S. Parsian, Q. Wu, B. Brisco, Google earth engine cloud computing platform for remote sensing big data applications: A comprehensive review, IEEE J. Sel. Top. Appl. Earth Obs. Remote. Sens. 13 (2020) 5326–5350, http://dx.doi.org/10.1109/JSTARS.2020.3021052, URL https://ieeexplore.ieee.org/document/9184118/.

[17] S. Subashini, V. Kavitha, A survey on security issues in service delivery models of cloud computing, J. Netw. Comput. Appl. 34 (1) (2011) 1–11, http://dx.doi.org/10.1016/j.jnca.2010.07.006, URL https://linkinghub.elsevier.com/retrieve/pii/S1084804510001281.

[18] S. Wei, X. Lin, F. Tu, Y. Wang, L. Liu, S. Yin, Reconfigurability, why it matters in AI tasks processing: A survey of reconfigurable AI chips, IEEE Trans. Circuits Syst. I. Regul. Pap. 70 (3) (2023) 1228–1241, http://dx.doi.org/10.1109/TCSI.2022.3228860, URL https://ieeexplore.ieee.org/document/9996124/.

[19] J. Melchert, K. Zhang, Y. Mei, M. Horowitz, C. Torng, P. Raina, Canal: A flexible interconnect generator for coarse-grained reconfigurable arrays, IEEE Comput. Archit. Lett. 22 (1) (2023) 45–48, http://dx.doi.org/10.1109/LCA.2023.3268126, URL https://ieeexplore.ieee.org/document/10105430/.

[20] C. Tan, C. Xie, A. Li, K.J. Barker, A. Tumeo, OpenCGRA: An open-source unified framework for modeling, testing, and evaluating CGRAs, ICCD, in: 2020 IEEE 38th International Conference on Computer Design, vol.2020-Octob, IEEE, 2020, pp. 381–388, http://dx.doi.org/10.1109/ICCD50377.2020.00070, URL https://ieeexplore.ieee.org/document/9283606/.

[21] C. Tan, C. Xie, T. Geng, A. Marquez, A. Tumeo, K. Barker, A. Li, ARENA: Asynchronous reconfigurable accelerator ring to enable data-centric parallel computing, IEEE Trans. Parallel Distrib. Syst. 32 (12) (2021) 2880–2892, http://dx.doi.org/10.1109/TPDS.2021.3081074, URL https://ieeexplore.ieee.org/document/9436051/.

[22] C. Tirelli, L. Ferretti, L. Pozzi, SAT-MapIt: A SAT-based modulo scheduling mapper for coarse grain reconfigurable architectures, in: 2023 Design, Automation & Test in Europe Conference & Exhibition, DATE, IEEE, 2023, pp. 1–6, http://dx.doi.org/10.23919/DATE56975.2023.10137123, URL https://ieeexplore.ieee.org/document/10137123/.

[23] Y. Zhuang, Z. Zhang, D. Liu, Towards high-quality CGRA mapping with graph neural networks and reinforcement learning, in: Proceedings of the 41st IEEE/ACM International Conference on Computer-Aided Design, ACM, New York, NY, USA, 2022, pp. 1–9, http://dx.doi.org/10.1145/3508352.3549458, URL https://dl.acm.org/doi/10.1145/3508352.3549458.

[24] D. Liu, S. Yin, G. Luo, J. Shang, L. Liu, S. Wei, Y. Feng, S. Zhou, Data-flow graph mapping optimization for CGRA with deep reinforcement learning, IEEE Trans. Comput.-Aided Des. Integr. Circuits Syst. 38 (12) (2019) 2271–2283, http://dx.doi.org/10.1109/TCAD.2018.2878183, URL https://ieeexplore.ieee.org/document/8509169/.

[25] S. Das, K. Martin, T. Peyret, P. Coussy, An efficient and flexible stochastic CGRA mapping approach, ACM Trans. Embed. Comput. Syst. 22 (1) (2023) 1–24, http://dx.doi.org/10.1145/3550071, URL https://dl.acm.org/doi/10.1145/3550071.

[26] Y. Fu, K. Chen, W. Song, G. He, S. Shen, H. Wang, C. Zhang, L. Li, A DSP-purposed reconfigurable acceleration machine (DREAM) for high energy efficiency MIMO signal processing, IEEE Trans. Circuits Syst. I. Regul. Pap. 70 (2) (2023) 952–965, http://dx.doi.org/10.1109/TCSI.2022.3220947, URL https://ieeexplore.ieee.org/document/9952172/.

[27] Y. Zhang, N. Zhang, T. Zhao, M. Vilim, M. Shahbaz, K. Olukotun, SARA: Scaling a reconfigurable dataflow accelerator, ISCA, in: 2021 ACM/IEEE 48th Annual International Symposium on Computer Architecture, vol. 2021-June, IEEE, 2021, pp. 1041–1054, http://dx.doi.org/10.1109/ISCA52012.2021.00085, URL https://ieeexplore.ieee.org/document/9499943/.

[28] J. Lee, J. Lee, NP-CGRA: Extending CGRAs for efficient processing of light-weight deep neural networks, DATE, in: 2021 Design, Automation & Test in Europe Conference & Exhibition, vol. 2021-Febru, IEEE, 2021, pp. 1408–1413, http://dx.doi.org/10.23919/DATE51398.2021.9474256, URL https://ieeexplore.ieee.org/document/9474256/.

[29] Y. Huang, P. Ienne, O. Temam, Y. Chen, C. Wu, Elastic CGRAs, in: Proceedings of the ACM/SIGDA International Symposium on Field Programmable Gate Arrays, ACM, New York, NY, USA, 2013, pp. 171–180, http://dx.doi.org/10.1145/2435264.2435296, URL https://dl.acm.org/doi/10.1145/2435264.2435296.

[30] A. Tumeo, N.B. Agostini, S. Curzel, A. Limaye, C. Tan, V. Amatya, M. Minutoli, V.G. Castellana, A. Li, J. Manzano, SO (DA)ˆ 2: End-to-end generation of specialized reconfigurable architectures (invited talk), in: 13th Workshop on Parallel Programming and Run-Time Management Techniques for Many-Core Architectures and 11th Workshop on Design Tools and Architectures for Multicore Embedded Computing Platforms (PARMA-DITAM 2022), vol. 100, Schloss Dagstuhl-Leibniz-Zentrum für Informatik, 2022, pp. 9–23, http://dx.doi.org/10.4230/OASIcs.PARMA-DITAM.2022.1.

[31] M. Vilim, A. Rucker, Y. Zhang, S. Liu, K. Olukotun, Gorgon: Accelerating machine learning from relational data, ISCA, in: 2020 ACM/IEEE 47th Annual International Symposium on Computer Architecture, vol. 2020-May, IEEE, 2020, pp. 309–321, http://dx.doi.org/10.1109/ISCA45697.2020.00035, URL https://ieeexplore.ieee.org/document/9138917/.

[32] Y. Luo, C. Tan, N.B. Agostini, A. Li, A. Tumeo, N. Dave, T. Geng, ML-CGRA: An integrated compilation framework to enable efficient machine learning acceleration on CGRAs, DAC, in: 2023 60th ACM/IEEE Design Automation Conference, vol. 2023-July, IEEE, 2023, pp. 1–6, http://dx.doi.org/10.1109/DAC56929.2023.10247873, URL https://ieeexplore.ieee.org/document/10247873/.

[33] C. Lattner, M. Amini, U. Bondhugula, A. Cohen, A. Davis, J. Pienaar, R. Riddle, T. Shpeisman, N. Vasilache, O. Zinenko, MLIR: Scaling compiler infrastructure for domain specific computation, in: 2021 IEEE/ACM International Symposium on Code Generation and Optimization, CGO, IEEE, 2021, pp. 2–14, http://dx.doi.org/10.1109/CGO51591.2021.9370308, URL https://ieeexplore.ieee.org/document/9370308/.

[34] S.A. Chin, N. Sakamoto, A. Rui, J. Zhao, J.H. Kim, Y. Hara-Azumi, J. Anderson, CGRA-ME: A unified framework for CGRA modelling and exploration, in: 2017 IEEE 28th International Conference on Application-Specific Systems, Architectures and Processors, ASAP, IEEE, 2017, pp. 184–189, http://dx.doi.org/10.1109/ASAP.2017.7995277, URL http://ieeexplore.ieee.org/document/7995277/.

[35] C. Tan, T. Tambe, J.J. Zhang, B. Fang, T. Geng, G.-Y. Wei, D. Brooks, A. Tumeo, G. Gopalakrishnan, A. Li, ASAP: automatic synthesis of area-efficient and precision-aware CGRAs, in: Proceedings of the 36th ACM International Conference on Supercomputing, ACM, New York, NY, USA, 2022, pp. 1–13, http://dx.doi.org/10.1145/3524059.3532359, URL https://dl.acm.org/doi/10.1145/3524059.3532359.

[36] C. Tan, C. Xie, A. Li, K.J. Barker, A. Tumeo, AURORA: Automated refinement of coarse-grained reconfigurable accelerators, in: 2021 Design, Automation & Test in Europe Conference & Exhibition, DATE, IEEE, 2021, pp. 1388–1393, http://dx.doi.org/10.23919/DATE51398.2021.9473955, URL https://ieeexplore.ieee.org/document/9473955/.

[37] J. Weng, S. Liu, V. Dadu, Z. Wang, P. Shah, T. Nowatzki, DSAGEN: Synthesizing programmable spatial accelerators, in: 2020 ACM/IEEE 47th Annual International Symposium on Computer Architecture, ISCA, IEEE, 2020, pp. 268–281,

http://dx.doi.org/10.1109/ISCA45697.2020.00032, URL https://ieeexplore.ieee.org/document/9138967/.

[38] J. Melchert, K. Feng, C. Donovick, R. Daly, R. Sharma, C. Barrett, M.A. Horowitz, P. Hanrahan, P. Raina, APEX: A framework for automated processing element design space exploration using frequent subgraph analysis, in: Proceedings of the 28th ACM International Conference on Architectural Support for Programming Languages and Operating Systems, vol. 3, ACM, New York, NY, USA, 2023, pp. 33–45, http://dx.doi.org/10.1145/3582016.3582070, URL https://dl.acm.org/doi/10.1145/3582016.3582070.

[39] O. Akbari, M. Kamal, A. Afzali-Kusha, M. Pedram, M. Shafique, PX-CGRA: Polymorphic approximate coarse-grained reconfigurable architecture, DATE, in: 2018 Design, Automation & Test in Europe Conference & Exhibition, vol. 2018-Janua, IEEE, 2018, pp. 413–418, http://dx.doi.org/10.23919/DATE.2018.8342045, URL https://ieeexplore.ieee.org/document/8342045/.

[40] Y.-H. Chen, T.-J. Yang, J.S. Emer, V. Sze, Eyeriss v2: A flexible accelerator for emerging deep neural networks on mobile devices, IEEE J. Emerg. Sel. Top. Circuits Syst. 9 (2) (2019) 292–308, http://dx.doi.org/10.1109/JETCAS.2019.2910232, URL https://ieeexplore.ieee.org/document/8686088/.

[41] Q.M. Nguyen, D. Sanchez, Fifer: Practical acceleration of irregular applications on reconfigurable architectures, in: MICRO-54: 54th Annual IEEE/ACM International Symposium on Microarchitecture, ACM, New York, NY, USA, 2021, pp. 1064–1077, http://dx.doi.org/10.1145/3466752.3480048, URL https://dl.acm.org/doi/10.1145/3466752.3480048.

[42] J. Qin, T. Xia, C. Tan, J. Zhang, S.Q. Zhang, PICACHU: Plug-in CGRA handling upcoming nonlinear operations in LLMs, ASPLOS '25, in: Proceedings of the 30th ACM International Conference on Architectural Support for Programming Languages and Operating Systems, vol. 2, Association for Computing Machinery, New York, NY, USA, 2025, pp. 845–861, http://dx.doi.org/10.1145/3676641.3716013, URL https://doi.org/10.1145/3676641.3716013.

[43] L. Zulberti, M. Monopoli, P. Nannipieri, L. Fanucci, S. Moranti, Highly parameterised CGRA architecture for design space exploration of machine learning applications onboard satellites, in: 2023 European Data Handling & Data Processing Conference, EDHPC, IEEE, 2023, pp. 1–6, http://dx.doi.org/10.23919/EDHPC59100.2023.10396632, URL https://ieeexplore.ieee.org/document/10396632/.

[44] L. Zulberti, M. Monopoli, P. Nannipieri, S. Moranti, G. Thys, L. Fanucci, Efficient coarse-grained reconfigurable array architecture for machine learning applications in space using DARE65T library platform, Microprocess. Microsyst. (2025) http://dx.doi.org/10.1016/j.micpro.2025.105142.

[45] G. Mystkowska, L. Zulberti, M. Monopoli, P. Nannipieri, L. Fanucci, Flexible precision vector extension for energy efficient coarse-grained reconfigurable array AI-engine, in: 2024 27th Euromicro Conference on Digital System Design, DSD, 2024, pp. 314–318, http://dx.doi.org/10.1109/DSD64264.2024.00049.

[46] L. Zulberti, M. Monopoli, P. Nannipieri, L. Fanucci, Architectural implications for inference of graph neural networks on CGRA-based accelerators, in: 2022 17th Conference on Ph.D Research in Microelectronics and Electronics, PRIME, IEEE, 2022, pp. 373–376, http://dx.doi.org/10.1109/PRIME55000.2022.9816810, URL https://ieeexplore.ieee.org/document/9816810/.

[47] L. Zulberti, A. Monorchio, M. Monopoli, G. Mystkowska, P. Nannipieri, L. Fanucci, SmartDMA: Adaptable memory access controller for CGRA-based processing systems, in: 2024 27th Euromicro Conference on Digital System Design, DSD, IEEE, 2024, pp. 306–313, http://dx.doi.org/10.1109/DSD64264.2024.00048, URL https://ieeexplore.ieee.org/document/10741788/.

[48] S. Nolting, et al., The neorv32 risc-v processor, 2022, Fischer, " The neorv32 risc-v processor.

[49] M. Capra, B. Bussolino, A. Marchisio, G. Masera, M. Martina, M. Shafique, Hardware and software optimizations for accelerating deep neural networks: Survey of current trends, challenges, and the road ahead, IEEE Access 8 (2020) 225134–225180, http://dx.doi.org/10.1109/ACCESS.2020.3039858, URL https://ieeexplore.ieee.org/document/9269334.

[50] M. Van de Burgwal, E.H. Boufouss, B. Vignon, L. Berti, G. Thys, M. Kakoulin, Validation, characterization and irradiation testing of the DARE65T platform, in: Proc. 9th Int. Workshop Analogue Mixed-Signal Integr. Circuits Space Appl., 2022, pp. 1–5.

[51] L. Zulberti, P. Nannipieri, L. Fanucci, A script-based cycle-true verification framework to speed-up hardware and software co-design of system-on-chip exploiting RISC-v architecture, in: 2021 16th International Conference on Design & Technology of Integrated Systems in Nanoscale Era, DTIS, IEEE, 2021, pp. 1–6, http://dx.doi.org/10.1109/DTIS53253.2021.9505139, URL https://ieeexplore.ieee.org/document/9505139/.

[52] A.G. Howard, M. Zhu, B. Chen, D. Kalenichenko, W. Wang, T. Weyand, M. Andreetto, H. Adam, MobileNets: Efficient convolutional neural networks for mobile vision applications, 2017, URL http://arxiv.org/abs/1704.04861.

[53] E. Dunkel, J. Swope, Z. Towfic, S. Chien, D. Russell, J. Sauvageau, D. Sheldon, J. Romero-Cañas, J.L. Espinosa-Aranda, L. Buckley, et al., Benchmarking deep learning inference of remote sensing imagery on the qualcomm snapdragon and intel movidius myriad x processors onboard the international space station, in: IGARSS 2022-2022 IEEE International Geoscience and Remote Sensing Symposium, IEEE, 2022, pp. 5301–5304.

[54] E.R. Dunkel, J. Swope, A. Candela, L. West, S.A. Chien, Z. Towfic, L. Buckley, J. Romero-Cañas, J.L. Espinosa-Aranda, E. Hervas-Martin, et al., Benchmarking deep learning models on myriad and snapdragon processors for space applications, J. Aerosp. Inf. Syst. 20 (10) (2023) 660–674.

Luca Zulberti received a Ph.D. in Information Engineering in 2025 from the Department of Information Engineering of the University of Pisa, where he also earned an M.Sc. degree in Electronic Engineering in 2021. His Ph.D., co-funded by the European Space Agency, focused on the topic "Innovative Coarse-Grained Reconfigurable Array Platform for Computing Artificial Intelligence On-Board". From March to October 2021, he worked as a research fellow at the Department of Information Engineering of the University of Pisa on the topic "Design of Embedded Systems based on RISC-V Architecture". His research interests are micro-architectural designs, embedded systems, and digital workflow automation.

Matteo Monopoli is a Ph.D. candidate in Information Engineering at the Department of Information Engineering of the University of Pisa. He received his B.Sc. and M.Sc. degrees in Electronic Engineering from the University of Pisa in 2020 and 2022, respectively. His Ph.D. has been co-funded by the European Space Agency on the topic "RISC-V Based SoC Featuring a Soft-GPU Hardware Accelerator for Artificial Intelligence On-board". He is currently a research fellow at the Department of Information Engineering of the University of Pisa. His research interests include reconfigurable and parallel computing platforms for accelerating AI algorithms in space applications, as well as reliability-aware micro-architectural design.

Gabriela Mystkowska is a Ph.D. student at the Department of Information Engineering at the University of Pisa. Her Ph.D. is co-funded by the European Space Agency on the topic "UDSM AI-Engine For Reliable, Energy-Efficient Next-Generation Satellites". She received her B.Sc. in Mechatronics and M.Sc. in Electronics Engineering from the Warsaw University of Technology (WUT). During her internship at ESA, she worked on SpaceFibre high-speed serial links. She has been a member of the Students' Astronautical Association at the WUT and participated in three student projects supported by ESA. Her background consists of high-speed digital electronics, electronic components production, and space environment testing.

Andrea Monorchio is a Ph.D. student at the Department of Information Engineering of the University of Pisa. He received his Bachelor's and Master's degrees in Electronic Engineering at the University of Pisa in November 2021 and April 2024, respectively. His M.Sc. thesis focused on the design and FPGA prototyping of a flexible Direct Memory Access Controller, contributing to a project aimed at developing a programming environment based on a Coarse-Grained Reconfigurable Array accelerator for executing time-bound AI algorithms. His research is currently focused on applications for AI algorithms in safety-critical systems, leveraging CGRAs as hardware accelerators.

Silvia Moranti got her degree in Electronic Engineering in 1998 from the University of Perugia. She is a Microelectronics engineer at ESA, supporting FPGA and ASIC development for several ESA missions, Earth Observation, like CHIME, CIMR, and ROSE-L, and Science, like JUICE, EUCLID, and PLATO. At the beginning of her career, she worked as an EMC Engineer at Angelantoni Industry. Then, she moved to NOKIA SIEMENS NETWORK as an FPGA designer for Base Band signal processing. She started to work in the space industry in 2008 at Thales Alenia Space as an FPGA designer, implementing FPGAs for several ESA and ASI missions.

Luca Fanucci got his Ph.D. degree in Electronic Engineering from the University of Pisa in 1996. From 1992 to 1996, he was with the European Space Agency as a research fellow. From 1996 to 2004, he was a senior researcher at the Italian National Research Council in Pisa. He is currently a Full Professor of Microelectronics at the University of Pisa. His research interests include integrated circuits and electronic systems design. He is co-author of more than 400 journal and conference papers and co-inventor of more than 40 patents. He served on several technical programme committees for international conferences.

Pietro Nannipieri got his Ph.D. degree in 2020 cum laude in Information Engineering from the University of Pisa. In 2019, Pietro spent several months as a Visiting Researcher in the TEC-EDP Section in ESTEC (ESA), where he carried out different qualification tests on the SpaceFibre technology. He is currently a Tenure-Track Assistant Professor in the Electronic Systems lab of the Department of Information Engineering at the University of Pisa. His work mainly focuses on developing IPs for satellite onboard processing and data handling (e.g., SpaceFibre), and hardware cryptography. His interests are digital and VLSI design, as well as electronics for space applications.